

AWS 3-Tier Infrastructure with Terraform: Step-by-Step Guide

For: complete beginners | **Default region:** Mumbai (`ap-south-1`) | **Tools:** Terraform, AWS CLI, a bash terminal

KEY TO NOTES IN THIS GUIDE

NOTE = good to know. **WARNING** = can cost money or break things. **ALTERNATIVE** = another way to do the same step.

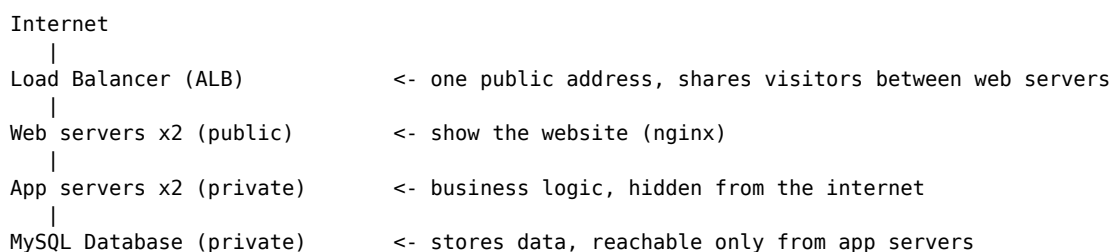
0. Replace these placeholders first

This guide uses placeholders instead of real names. Pick your own values and use them everywhere:

Placeholder	Meaning	Example
<code><UNIQUE_BUCKET_NAME></code>	Your S3 bucket for Terraform state. Must be unique worldwide, lowercase	<code>mycompany-tf-state-2026</code>
<code><KEY_PAIR_NAME></code>	Name of your AWS key pair (without <code>.pem</code> or <code>.ppk</code>)	<code>training-key</code>
<code><PROJECT_PATH></code>	Folder where you unzip the project	<code>/c/Terraform/terraform-3tier</code>
<code><WEB_PUBLIC_IP> / <APP_PRIVATE_IP></code>	Server addresses, copied from the EC2 console	shown after deploy

1. What are we building?

A website setup the way real companies build it, in three layers (tiers):



Everything is created by **code** (Terraform) instead of clicking in the AWS Console. One command builds it, one command deletes it, and we can make four copies: **dev**, **qa**, **uat**, **prod**.

2. Words you must know

Word	Simple meaning
AWS	Amazon's cloud: rent servers by the hour
Terraform	A tool that reads code files and builds AWS resources for you
VPC	Your own private network inside AWS
Subnet	A section of the VPC (public = internet access, private = no direct access)
Internet Gateway	The door between the VPC and the internet

Word	Simple meaning
Security Group	A firewall: who may connect to a server, and on which port
EC2	A virtual server
Launch Template	The recipe for a server (image, size, key, firewall)
Auto Scaling Group (ASG)	Keeps the right number of servers running; replaces broken ones
ALB	Application Load Balancer: spreads traffic over servers
RDS	Managed database service (we use MySQL)
Module	A reusable folder of Terraform code (one per component)
Workspace	A separate copy of state so dev, qa, uat, prod don't mix
State file	Terraform's notebook of what it has built (kept in S3)
tfvars file	Values for one environment
Key pair	Lock-and-key for logging in to servers (.pem or .ppk)

3. Prerequisites (do once)

3.1 AWS account and access keys

1. Log in to the AWS Console. Create an **IAM user** (training: **AdministratorAccess**).
2. IAM, then Users, then your user, then **Security credentials**, then **Create access key**. Save both values.

WARNING: Never share, screenshot or commit your access keys. Do not use the root account for daily work.

3.2 Install the tools

Windows: install **Git for Windows** (gives you Git Bash), **AWS CLI v2**, and **Terraform** (unzip **terraform.exe** into a folder and add it to the Windows **PATH**).

ALTERNATIVE (Linux / macOS): use your package manager, for example **brew install terraform awscli** on macOS, or the official HashiCorp and AWS install pages on Linux. On Windows you can also use **WSL** (Ubuntu inside Windows).

Open a **new** terminal and check:

```
terraform -version    # 1.5 or higher
aws --version
```

3.3 Connect the AWS CLI to your account

```
aws configure
```

Enter: Access key ID, Secret access key, region **ap-south-1**, output **json**. Test:

```
aws sts get-caller-identity
```

It should print your account number.

3.4 Key pair (to log in to servers)

EC2 Console, then **Key Pairs**, then **Create key pair** (in the same region you deploy to). The code uses it by **name**.

ALTERNATIVE: choose the **.pem** format when creating the key and you can skip the PuTTY conversion in Section 8. Choose **.ppk** only if you use PuTTY.

NOTE: Terraform never reads the key file. It only needs the key pair **name**. Set it in **variables.tf: default = "<KEY_PAIR_NAME>"**

4. One-time setup: where Terraform stores its state

Terraform needs an S3 bucket (stores state) and a DynamoDB table (stops two people running at once).

```
aws s3api create-bucket --bucket <UNIQUE_BUCKET_NAME> --region ap-south-1 \
--create-bucket-configuration LocationConstraint=ap-south-1

aws s3api put-bucket-versioning --bucket <UNIQUE_BUCKET_NAME> \
--versioning-configuration Status=Enabled

aws dynamodb create-table --table-name terraform-lock-table \
--attribute-definitions AttributeName=LockID,AttributeType=S \
--key-schema AttributeName=LockID,KeyType=HASH \
--billing-mode PAY_PER_REQUEST --region ap-south-1
```

Then open **backend.tf** and set **bucket = "<UNIQUE_BUCKET_NAME>"**.

ALTERNATIVE 1 (Console): create the bucket in S3 (enable Versioning) and a DynamoDB table named **terraform-lock-table** with partition key **LockID** (type String).

ALTERNATIVE 2 (practice only, no S3): delete or comment out the whole **backend "s3" {...}** block in **backend.tf**. Terraform then keeps state in a local file. **WARNING:** local state is lost if the file is deleted and cannot be shared with a team.

5. Understanding the project

terraform-3tier/	
+-- provider.tf	which cloud (AWS) and versions
+-- backend.tf	where the state file is stored (S3 + DynamoDB)
+-- variables.tf	inputs (region, environment, vpc_cidr, key_name...)
+-- main.tf	calls every module and connects them
+-- outputs.tf	values printed at the end (website address, DB address)
+-- environments/	dev.tfvars, qa.tfvars, uat.tfvars, prod.tfvars
+-- modules/	
+-- vpc/	network, 6 subnets, internet gateway, route table
+-- security-groups/	firewalls for ALB, web, app, database
+-- alb/	load balancer, target group, listener
+-- web/	launch template + Auto Scaling group (web)
+-- app/	launch template + Auto Scaling group (app)
+-- rds/	database + database subnet group

Each module has **main.tf** (what to build), **variables.tf** (inputs) and **outputs.tf** (results).

Traffic rules (who may talk to whom):

- Internet reaches only the **ALB** (port 80).
- Only the ALB reaches **web servers** (port 80).
- Only web servers reach **app servers** (port 8080 and SSH 22).
- Only app servers reach the **database** (port 3306).

NOTE (other regions): the code uses **ap-south-1a** and **ap-south-1b** in **modules/vpc/main.tf** and **ap-south-1** in the tfvars and **backend.tf**. To use another region, change all of those.

6. Deploy the dev environment

```
cd <PROJECT_PATH>
```

Step 1: Initialize (downloads the AWS plugin, connects to S3):

```
terraform init
```

Success message: *Terraform has been successfully initialized!*

Step 2: Create the workspace (first time only):

```
terraform workspace new dev
terraform workspace show      # prints: dev
```

If it exists already: `terraform workspace select dev`.

Step 3: Preview (creates nothing):

```
terraform plan -var-file=environments/dev.tfvars
```

Look for: *Plan: N to add, 0 to change, 0 to destroy*.

Step 4: Build:

```
terraform apply -var-file=environments/dev.tfvars
```

Type **yes**. Wait 10 to 15 minutes (the database is the slowest).

Step 5: Results:

```
terraform output
```

WARNING (cost): servers, load balancer and database are billed while they exist. Destroy when finished (Section 10).

ALTERNATIVE: `terraform apply -auto-approve -var-file=...` skips the **yes** question. Use only when you are sure.

7. Verify everything works

1. **Website:** `curl http://<alb_dns_name>` should show **Web server - ...**. If it fails, wait 2 to 3 minutes (the load balancer needs time to mark servers healthy).
2. **Console checks:** VPC **dev-vpc**; EC2 shows 2 **dev-web** and 2 **dev-app**; Load Balancers shows **dev-alb** with *healthy* targets; RDS shows **dev-db** Available.

ALTERNATIVE: open `http://<alb_dns_name>` in a browser instead of using `curl`.

8. Log in to the servers

The user name is always **ec2-user** (Amazon Linux). The web servers have a public IP; the app servers are private.

Option A: bash / Linux / macOS (needs a .pem key):

```
chmod 400 <KEY_PAIR_NAME>.pem

# Web server
ssh -i <KEY_PAIR_NAME>.pem ec2-user@<WEB_PUBLIC_IP>
```

```
# App server (private): jump through the web server
ssh -i <KEY_PAIR_NAME>.pem -J ec2-user@<WEB_PUBLIC_IP> ec2-user@<APP_PRIVATE_IP>
```

Option B: PuTTY (Windows, uses the .ppk key): Host = <WEB_PUBLIC_IP>, Connection, then SSH, then Auth, then Credentials, then pick the .ppk. For app servers, log in to the web server first, then SSH onward (enable agent forwarding with Pageant).

Option C: convert .ppk to .pem: PuTTYgen, Load the .ppk, Conversions, Export OpenSSH key.

ALTERNATIVE (no key needed for web servers): EC2 Console, select the instance, **Connect**, **EC2 Instance Connect**.

NOTE: the database is private. Connect to it only from an app server.

9. Build the other environments (qa, uat, prod)

```
terraform workspace new qa
terraform apply -var-file=environments/qa.tfvars
```

IMPORTANT RULE: the workspace name and the tfvars file must be the same environment. Always run `terraform workspace show` before `apply`.

10. Delete everything

```
terraform workspace select dev
terraform destroy -var-file=environments/dev.tfvars
```

Type **yes**. The S3 bucket and DynamoDB table stay (they cost almost nothing).

WARNING: `destroy` deletes the database and its data. Never run it against a real production environment by mistake.

11. Common errors and fixes

Error	Cause	Fix
<code>terraform: command not found</code>	Not in PATH	Add its folder to PATH, open a new terminal
Asks for <code>var.environment</code>	tfvars not passed	Add <code>-var-file=environments/dev.tfvars</code>
Invalid CIDR	Typed a wrong range	Use <code>10.0.0.0/16</code>
Backend or bucket not found	Section 4 skipped, or wrong bucket name	Create the bucket; fix <code>backend.tf</code>
<code>MasterUserPassword is not a valid password</code>	Password contains @ / " or space	Use a password such as <code>Password#123</code>
<code>InvalidKeyPair.NotFound</code>	Key pair name wrong or in another region	Fix <code>key_name</code> ; check the region
Instances have no name	Missing Name tag	Needs the <code>tag {}</code> block in the web and app ASGs (already included)
<code>AlreadyExists</code> for load balancer or DB	Same name used twice	Names include the environment prefix

Error	Cause	Fix
SSH timeout	Port 22 closed or wrong IP	Check the security group; use the instance's public IP
SSH Permission denied	Wrong key or user	Use the right key, user ec2-user , chmod 400
Website not loading after apply	Targets not healthy yet	Wait 2 to 3 minutes

12. Good practices and next steps

WARNING (security, read before sharing or reusing):

- The database password in `modules/rds/main.tf` is a **training value written in code**. Never do this in real projects.
 - **ALTERNATIVE (better)**: use a variable marked `sensitive = true`, or set `manage_master_user_password = true` so AWS creates and stores the password in Secrets Manager.
 - SSH is open to everyone (`0.0.0.0/0`) for training. Restrict `ssh_cidr` in `variables.tf` to `<your-ip>/32`.
- Private app servers have no internet (no NAT gateway). Add one if they must download software.
 - Production upgrades: HTTPS certificate, multi-AZ database, CloudWatch monitoring, backups, tags on all resources.

13. Command cheat-sheet

<code>terraform init</code>	<code># once per folder</code>
<code>terraform workspace new dev</code>	<code># create environment</code>
<code>terraform workspace select dev</code>	<code># switch environment</code>
<code>terraform workspace show</code>	<code># which one am I in?</code>
<code>terraform fmt</code>	<code># tidy the code</code>
<code>terraform validate</code>	<code># check the code</code>
<code>terraform plan -var-file=environments/dev.tfvars</code>	<code># preview</code>
<code>terraform apply -var-file=environments/dev.tfvars</code>	<code># build</code>
<code>terraform output</code>	<code># show addresses</code>
<code>terraform destroy -var-file=environments/dev.tfvars</code>	<code># delete all</code>